

UNIT 5 · WEEK 7

Pipes and process composition

📅 Week 7 · dates to be confirmed </> Lab TD4 (solved + AI challenge) ~ 2 animations

MANUAL M5 · THEORY OF THE UNIT

Pipes and process composition

Core reading for unit 5 (≈ 1 h). The outputs are real: TD4 programs run on the course machine.

CONTENTS

- | | | | |
|----|--|----|--|
| §1 | The pipe: a buffer with two ends | §3 | Redirecting with <code>dup2</code> and executing |
| §2 | The rules: blocking, end of file and SIGPIPE | §4 | Pipelines of N commands |
| | | §5 | Beyond |

§1 The pipe: a buffer with two ends

`pipe(fds)` creates a circular buffer in the kernel and returns two descriptors: `fds[0]`, the **read** end, and `fds[1]`, the **write** end. What is written into one is read from the other, in order, as a byte stream without message boundaries. The pipe has no name (unlike a FIFO, unit 3): only processes that inherit its descriptors can use it, so it is created **before** `fork`. Afterwards, each process **closes the end it does not use**: the writing parent closes `fds[0]`, the reading child closes `fds[1]`.

C tds/td4/mypipe.c

```
/*
 * TD4 - exercises 1 and 2: an unnamed pipe between a parent (writer) and a child
 (reader).
 * ./mypipe [count]      the parent writes "Hello World\n" `count` times (default 10)
 * The child reads AT MOST 500 bytes, once, prints them and leaves.
 *
 * Exercise 2: with a large count (10000 x 12 bytes = 120 KB > 64 KiB, the pipe
 capacity)
 * the parent blocks on write() once the pipe is full. When the child exits, nobody can
 read
 * any more: the next write() raises SIGPIPE and kills the parent ("Broken pipe", status
 141
 * in the shell). That only works if the parent CLOSED its own read end: with
 * -DKEEP_READ_END (Makefile target mypipe_bug) the parent itself is still a potential
 reader,
 * the kernel never raises SIGPIPE and the parent blocks forever.
```

```

*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

#define MESSAGE    "Hello World\n"
#define READ_MAX   500

int main(int argc, char *argv[])
{
    int count = argc > 1 ? atoi(argv[1]) : 10;
    int fds[2];                                /* fds[0] read end, fds[1] write
end */

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                             /* child: reader */
        exit_if(close(fds[1]) == -1, "close");    /* never writes */
        char buffer[READ_MAX];
        ssize_t n = read(fds[0], buffer, READ_MAX); /* one single read */
        exit_if(n == -1, "read");
        exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n) == -1, "write");
        fprintf(stderr, "I am the child, I read %zd bytes and I leave\n", n);
        exit_if(close(fds[0]) == -1, "close");
        return EXIT_SUCCESS;
    }

    /* parent: writer */
#ifdef KEEP_READ_END
    exit_if(close(fds[0]) == -1, "close");        /* THE line that exercise 2 is
about */
#endif
    for (int i = 0; i < count; i++)
        exit_if(write_all(fds[1], MESSAGE, strlen(MESSAGE)) == -1, "write");
    fprintf(stderr, "I am the parent, I wrote %d messages and I leave\n", count);
    exit_if(close(fds[1]) == -1, "close");
    exit_if(waitpid(pid, NULL, 0) == -1, "waitpid");
    return EXIT_SUCCESS;
}

```

Real output

```
$ ./mypipe
I am the parent, I wrote 10 messages and I leave
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
I am the child, I read 120 bytes and I leave
```

Practice this → TD4 · 1

§2 The rules: blocking, end of file and SIGPIPE

- **Reading** from an empty pipe blocks until data arrives... unless there is **no** write descriptor left open in any process: then `read` returns 0, end of file.
- **Writing** to a full pipe blocks until someone reads. The default capacity on Linux is 64 KiB (65,536 bytes); it is queried and changed with `fcntl(F_GETPIPE_SZ / F_SETPIPE_SZ)`.
- Writing when there is **no reader** left raises the `SIGPIPE` signal, which kills the process by default (status 141 = 128 + 13 in the shell); if it is ignored, `write` fails with `EPIPE`.
- A write of up to `PIPE_BUF` = 4096 bytes is **atomic**: it is not interleaved with other writers'.

TD4 · 2 shows the first three rules at once. The child reads 500 bytes once and ends; the parent writes N 12-byte messages. If the parent finishes before the child leaves, all goes well; if the child leaves first, the parent's next `write` gets `SIGPIPE`. With many messages this always happens, but in between it is a **race** between two processes running on different cores: the outcome changes from run to run. Measured 30 times per size:

Real output

```
$ for n in 50 100 ... 1000; do run ./mypipe n 30 times; count exit statuses; done
50 messages: 30 x status 0      0 x status 141 (SIGPIPE)
100 messages: 30 x status 0     0 x status 141 (SIGPIPE)
200 messages: 28 x status 0     2 x status 141 (SIGPIPE)
300 messages: 19 x status 0    11 x status 141 (SIGPIPE)
400 messages:  5 x status 0    25 x status 141 (SIGPIPE)
500 messages:  5 x status 0    25 x status 141 (SIGPIPE)
1000 messages:  0 x status 0   30 x status 141 (SIGPIPE)
```

If the parent forgets to close `fds[0]`, it counts as a reader itself: it never gets `SIGPIPE`, fills the 64 KiB and blocks forever waiting for "someone" to read:

Real output

```
$ timeout 3 ./mypipe_bug 10000 > /dev/null; echo "exit status: $? (124 = killed by
timeout: it was blocked)"
I am the child, I read 500 bytes and I leave
exit status: 124 (124 = killed by timeout: it was blocked)
```

Practice this → A5.2 capacity and race · TD4 · 2

§3 Redirecting with `dup2` and executing

`dup2(fds[1], STDOUT_FILENO)` makes descriptor 1 point to the pipe (unit 3: it copies the pointer to the open file table entry). Since `exec` keeps the descriptors (unit 4), the program executed afterwards will write into the pipe without knowing it. That is how other people's programs, like `ps`, are connected to a reader of our own:

C tds/td4/upper.c

```
int main(void)
{
    int fds[2];

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                /* child */
        exit_if(close(fds[1]) == -1, "close"); /* otherwise it never sees end of file */
        to_upper(fds[0], STDOUT_FILENO);
        return EXIT_SUCCESS;
    }

    /* parent */
    exit_if(close(fds[0]) == -1, "close");
    exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2");
    exit_if(close(fds[1]) == -1, "close"); /* 1 already refers to the pipe */
    execlp("ps", "ps", (char *)NULL);      /* "ps" twice: file to run, then
    argv[0] */
    exit_if(1, "execlp ps");
    return EXIT_FAILURE;
}
```

Real output

```
$ ./upper | head -4
PID TTY          TIME CMD
6706 ?           00:00:25 UVICORN
9771 ?           00:00:03 SYSTEMD
9801 ?           00:00:00 (SD-PAM)
```

Practice this → TD4 · 3

§4 Pipelines of N commands

For $c_1 | c_2 | \dots | c_N$ the shell creates $N - 1$ pipes and N children. The algorithm fits in a loop with an `in` variable that keeps the read end of the previous pipe: for each command, if it is not the last, `pipe`; `fork`; the child does `dup2(in, 0)` and `dup2(fds[1], 1)`, closes everything else and calls `exec`; the parent closes `in` and `fds[1]` and keeps `fds[0]` as the new `in`. At the end, it

waits for all of them and returns the status of the **last** one (in bash, `set -o pipefail` returns the one of the last that failed).

The golden rule: **every pipe must end up with exactly one write descriptor and one read descriptor**, in the two processes that use it. A single forgotten `close` —in the parent or in a child that inherited the pipe— leaves an extra writer and the last command waits for an end of file that never comes.

C tds/td4/pipeline_args.c

```
int in = STDIN_FILENO;
pid_t last = -1;
for (int i = 0; i < ncommands; i++) {
    int fds[2] = { -1, -1 };
    if (i < ncommands - 1)
        exit_if(pipe(fds) == -1, "pipe");

    pid_t pid = fork();
    exit_if(pid == -1, "fork");
    if (pid == 0) {
        if (in != STDIN_FILENO) {
            exit_if(dup2(in, STDIN_FILENO) == -1, "dup2 stdin");
            exit_if(close(in) == -1, "close");
        }
        if (fds[1] != -1) {
            exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2 stdout");
            exit_if(close(fds[0]) == -1 || close(fds[1]) == -1, "close");
        }
        execvp(commands[i][0], commands[i]);
        perror(commands[i][0]);
        _exit(127);
    }
    if (in != STDIN_FILENO)
        exit_if(close(in) == -1, "close");
    if (fds[1] != -1) {
        exit_if(close(fds[1]) == -1, "close");
        in = fds[0];
    }
    last = pid;
}
```

Real output

```
$ ./pipeline_args ls -l demo -- grep txt -- wc -l
2
```

Real output

```
$ ./pipeline_args ls -- sh -c 'kill -TERM $$'
pipeline: last command killed by SIGTERM
[exit status 1]
```

Practice this → A5.1 building a pipeline · TD4 · 4 and 5

§5 Beyond

- `pipe2(fds, O_CLOEXEC)` creates the pipe with the descriptors marked to close on `exec`: it prevents a launched program from inheriting ends that do not belong to it (and that, as in §4, would prevent an end of file).
- `splice` and `tee` move data between a pipe and a file or socket inside the kernel, without copying it into the process's space.
- **Process substitution** in bash (`diff <(ls a) <(ls b)`): the shell creates pipes and passes them as `/dev/fd/N` paths.
- **Two-way** communication needs two pipes or a `socketpair` (unit 8).